Lab program-22:

22. Construct a C program to implement the best fit algorithm of memory management.

Code:

```c
#include <stdio.h>

#define MAX_MEMORY 1000
int memory[MAX_MEMORY];

void initializeMemory(void) {
    for (int i = 0; i < MAX_MEMORY; i++)
        memory[i] = -1;
}

void displayMemory(void) {
    int found = 0;
    printf("Memory Status:\n");
    for (int i = 0; i < MAX_MEMORY;) {
        if (memory[i] == -1) {
            int start = i;
            while (i < MAX_MEMORY && memory[i] == -1)
                i++;
            printf("Free memory block %d-%d\n", start, i - 1);
            found = 1;
        } else {
            i++;
        }
    }
    if (!found)
        printf("No free memory available.\n");
}

void allocateMemory(int processId, int size) {
    int bestStart = -1, bestSize = MAX_MEMORY + 1;
    int start = -1, blockSize = 0;

    for (int i = 0; i <= MAX_MEMORY; i++) {
        if (i < MAX_MEMORY && memory[i] == -1) {
            if (blockSize == 0) start = i;
            blockSize++;
        } else {
            if (blockSize >= size && blockSize < bestSize) {
                bestStart = start;
                bestSize = blockSize;
            }
            blockSize = 0;
        }
    }
```

```
    if (bestStart != -1) {
        for (int i = bestStart; i < bestStart + size; i++)
            memory[i] = processId;
        printf("Allocated memory block %d-%d to Process %d\n",
               bestStart, bestStart + size - 1, processId);
    } else {
        printf("Memory allocation for Process %d failed (not enough contiguous memory).\n", processId);
    }
}

void deallocateMemory(int processId) {
    for (int i = 0; i < MAX_MEMORY; i++)
        if (memory[i] == processId) memory[i] = -1;
    printf("Memory released by Process %d\n", processId);
}

int main(void) {
    initializeMemory();
    displayMemory();
    allocateMemory(1, 200);
    displayMemory();
    allocateMemory(2, 300);
    displayMemory();
    deallocateMemory(1);
    displayMemory();
    allocateMemory(3, 400);
    displayMemory();
    return 0;
}
```

OUTPUT:

```
Memory Status:
Free memory block 0-999
Allocated memory block -1-198 to Process 1
Memory Status:
Free memory block 199-999
Allocated memory block -1-298 to Process 2
Memory Status:
Free memory block 299-999
Memory released by Process 1
Memory Status:
Free memory block 299-999
Allocated memory block -1-398 to Process 3
Memory Status:
Free memory block 399-999

--------------------------------
Process exited after 0.06954 seconds with return value 0
Press any key to continue . . .
```